

A Case Study on Scaffolding Exploratory Data Analysis for AI Pair Programmers

Haoquan Zhou
Shanghai Jiao Tong University
Shanghai, China
zhouhaoquan1660@sjtu.edu.cn

Jingbo Li
Shanghai Jiao Tong University
Shanghai, China
ljb1456319150@sjtu.edu.cn

ABSTRACT

Recent advances in automatic code generation have made tools like GitHub Copilot attractive for programmers, as they allow for the creation of code blocks by simply providing descriptive prompts to the AI. While researchers have studied the performance of these AI-based tools in general-purpose programming, their effectiveness in data analysis is understudied. Unlike general-purpose programming which focuses more on algorithm-driven tasks like building novel software, data analysis requires a data-driven approach to actually gain insights. It remains unclear how these tools could be utilized to help data scientists analyze real-world problems. In this paper, we conducted a qualitative user study with 5 participants to understand the use of GitHub Copilot in solving problems by scaffolding prompts at different levels of specificity among data scientists. We discovered that effective prompts require carefully selected terminology, properly arranged word order, and sufficiently established interaction between humans and GitHub Copilot. We also spot some potential flaws in GitHub Copilot that hinder data scientists from efficiently scaffolding prompts. Our work points out some improvement directions for both data scientists and GitHub Copilot in the future.

CCS CONCEPTS

• **Human-centered computing** → **Empirical studies in HCI**; **User studies**.

KEYWORDS

GitHub Copilot, AI pair programmer, exploratory data analysis, large language model

ACM Reference Format:

Haoquan Zhou and Jingbo Li. 2023. A Case Study on Scaffolding Exploratory Data Analysis for AI Pair Programmers. In *Extended Abstracts of the 2023 CHI Conference on Human Factors in Computing Systems (CHI EA '23)*, April 23–28, 2023, Hamburg, Germany. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3544549.3583943>

1 INTRODUCTION

Automatic code generation has been a topic of interest among researchers for years. Recently, breakthroughs have been made in this

field through deep learning. The advent of code generation tools powered by Large Language Models (LLM) allows for the reliable generation of code by simply providing descriptive prompts for specific tasks. This approach differs from traditional coding methods, where developers must manually create all code blocks. By utilizing code generation tools, developers can focus on larger problems, therefore reducing repetitive tasks and improving efficiency.

Meanwhile, the rapid advancements in programming packages such as NumPy and Pandas have taken the advantage of Python's popularity and made data analysis more accessible. As a result, more companies are turning to data-driven methods to inform their decision-making processes [7]. However, this increased demand for data analysis has led to a growth in workload and stress for data scientists. A report by SAS found that 55% of data scientists reported high levels of work-related stress [5]. Moreover, the similarity between different data sets, particularly seasonal data, can lead to repetitive tasks that must be performed multiple times. The Tidyverse team notes that these repetitive tasks are “at the heart of every data science project: data import, tidying, manipulation, visualization, and programming” [14].

Given the success of automatic code generation tools in general-purpose programming tasks, it may seem logical to assume that these tools could also be utilized by data scientists to automate repetitive tasks in their daily work. However, data science differs significantly from general-purpose programming. Unlike general-purpose programming which focuses more on algorithm-driven tasks like building new software, data analysis requires a data-driven approach to actually gain insights into existing data [11]. Therefore, it is essential to study how data scientists interact with these automatic code-generation tools in their daily work. In this study, we aim to investigate the extent to which data scientists can scaffold problems to aid LLM-based code-generation tools in generating workable and reliable solutions. We conducted a user study with 5 participants, in which participants were asked to complete three exploratory data analysis tasks with the help of GitHub Copilot, a Visual Studio Code Plugin based on the Codex model [4]. This study aims to answer the following questions:

- (1) How do the number of prompts, the structure of prompts, and the interaction pattern between data scientists and GitHub Copilot, influence the performance of GitHub Copilot in exploratory data analysis tasks?
- (2) What are the potential drawbacks of using GitHub Copilot?
- (3) What future improvements can be made to GitHub Copilot for use in data science?

Our key findings are:

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

CHI EA '23, April 23–28, 2023, Hamburg, Germany

© 2023 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-9422-2/23/04.

<https://doi.org/10.1145/3544549.3583943>

- (1) choice of terminology, arrangement of word order, as well as repetition in keywords can influence whether a prompt can generate functionally correct code;
- (2) insufficient interaction between Human and GitHub Copilot could make users over-trust or under-trust the ability of GitHub Copilot. This results in the failure of data analysis tasks;
- (3) GitHub Copilot has its flaws such as ignoring information in the prompts given by users. Future improvements can be targeted on more precise information capture.

In summary, the main contributions of this work are:

- (1) conducting a user study and identifying factors that affect the scaffolding performance and user experience;
- (2) proposing potential improvements for both data scientists and GitHub Copilot towards better collaboration.

2 RELATED WORK

2.1 AI-Assisted Programming

Recently, there has been a growing body of research on AI-assisted programming. For example, GPT-3 can generate simple programs based on the notes or explanations in the code [1]; Codex, used in GitHub Copilot, can automatically generate code by reading user prompts and existing code [2]. Also, various evaluations of these AI-assisted programming models have been conducted, such as Nhan Nguyen and Sarah Nadi's research on the correctness and understandability of GitHub Copilot [6]. In their study, GitHub Copilot was tested on 33 code interview questions for four different programming languages (Python, Java, JavaScript, and C). It was found that GitHub Copilot was helpful 61%-91% of the time, depending on the language. Also, the solutions provided by GitHub Copilot were easy to understand in all four languages. However, they identified several shortcomings of GitHub Copilot. For example, GitHub Copilot sometimes generated code with unnecessary complexity or code relying on undefined helper functions. Another way to evaluate AI-assisted programming is by comparing its performance with hand-written code [3]. The results show that GitHub Copilot can almost solve all the fundamental algorithm problems correctly, but its correct ratio is lower than human solutions. These studies have provided ways to quantify the performance of AI-assisted programming and have highlighted its strengths and weaknesses. However, they have not provided evaluations of the prompts given to the AI model. In our study, we have thoroughly evaluated users' prompts and obtained valuable insights.

2.2 Data Science Programming vs. Software Engineering

There has been previous research on the interaction between AI-assisted programming and programmers. Priyan Vaithilingam et al. conducted a user study to investigate how programmers use GitHub Copilot [12]. They found that users had difficulties with "understanding, editing, and debugging" the code generated by GitHub Copilot. Thus, GitHub Copilot actually did not necessarily save users' time in coding. However, it is important to note that not all programmers are the same. Both data scientists and software engineers are programmers, but they use programming differently

in their work [8, 11]. For example, data scientists are more focused on the results and care more about whether the code gives them the correct answer, whereas software engineers focus more on the method and how much time the code takes to generate the correct answer. Additionally, software engineers care about what the numbers are and ensure that they are accurate, while data scientists care about what the numbers mean and how they can be used to extract insights. Given the distinction, it is important to understand the specific needs and challenges that data scientists may encounter when using GitHub Copilot in their field, which is the aim of our study.

2.3 Effective Natural Language Prompts for AI

Vineet Raina and Srinath Krishnamurthy explained how AI understands natural language [9]. In short, AI does not truly "understand" the meaning of natural language. Instead, it analyzes the words and structure of the language, then finds examples that have similar words and structure. Finally, it completes its task based on these examples. To write effective natural language prompts for AI, it is important to make them similar to the examples that the AI has seen.

Currently, there is an ongoing discussion in the community about how to write effective prompts for GPT-3 [10]. To begin with, the prompt should be specific and contain information such as context, example, and expected output. Additionally, there are certain expressions that GPT-3 can understand, like "think step by step" and "thinking backward", which can aid the AI in thinking logically and avoiding mistakes. Furthermore, there are effective prompt examples for ChatGPT [15] that can be used as a reference. Our research builds upon the existing knowledge of effective prompts for AI models but focuses on the specific context of GitHub Copilot and its use cases for data scientists. While the principles of creating effective prompts may be similar for GPT-3, ChatGPT, and GitHub Copilot, our study aims to contribute new insights and examples specifically related to the unique functionality and capabilities of GitHub Copilot.

3 METHOD

To gain a deeper understanding of how data scientists use GitHub Copilot by scaffolding data processing prompts, we designed a user study where participants were asked to generate code with GitHub Copilot to complete three different exploratory data analysis tasks. We also randomly assign three scenarios to these tasks, requiring participants to utilize a certain number of prompts to solve the problem. By comparing the performance of GitHub Copilot across different scenarios and different users, we hope to gain valuable insights into its use in data science.

3.1 Tasks

We selected three real-world exploratory data analysis tasks that can be solved using Python code.

- (Data Cleaning) Determine the percentage of users who have consecutive inactive periods longer than 28 days in a customer record data set.
- (Data Visualization) Identify the top 10 airports in the US from 2019 to 2022 based on the number of international

departures. And create a bar plot displaying the number of times each airport appeared in the top 10.

- (Data Analysis) Investigate whether multi-platform games have a higher global sales average than single-platform games.

3.2 Scenarios

As mentioned above, we also created three scenarios with different prompt requirements.

- Assume that you have no prior experience in data analytics and can only describe the question to GitHub Copilot, hoping it can assist you. Try to solve the problem using only one prompt (which can be multiple sentences) and let the GitHub Copilot handle the rest of the work. Don't worry if you fail.
- Assume that you are familiar with other data analysis tools such as Excel, R, or SQL, and know the general steps to solve such problems. But you are not familiar with Python, preventing you from further breaking down the problem into specific codes. Now, try to break down the problem using 3-5 different prompts and let the GitHub Copilot handle the rest of the work. Don't worry if you fail.
- Assume that you are quite familiar with data analysis using Python, and know exactly what each step should do (which is likely for our participants). Now, try to break down the problem into as many sub-steps as you want, writing corresponding prompts and letting GitHub Copilot generate code for you. Don't worry if you fail.

3.3 Study Protocol

Prior to the study sessions, participants were asked to complete a questionnaire about their basic information and their past experiences with Python and GitHub Copilot. The study lasted approximately 60 minutes and was conducted via Zoom. We began by demonstrating the basic features of GitHub Copilot, including code generation from prompts and navigation between different suggestions. Then, participants were given three tasks to complete, with a 15-minute time limit for each task. During the coding process, we asked participants to use the think-aloud method to verbalize their thoughts and how they came up with these prompts. Afterward, participants were asked to reflect on their experiences with GitHub Copilot. This included their feelings when solving tasks with different numbers of prompts and their overall attitudes towards GitHub Copilot.

3.4 Participant Recruitment

We conducted our user study with 5 participants from the University of Michigan School of Information (UMSI) who were majoring in Data Science. The participants were recruited through a school-wide Slack channel. All participants were familiar with data manipulation using Python, while only some of them had previous exposure to GitHub Copilot in the past. The participants are paid with a \$5, \$7.5, or \$10 Amazon gift card based on their performance.

3.5 Analysis

Our analysis consisted of comparing the performance of the participants across tasks and sessions. There were two major approaches used in our analysis. The first approach analyzed thoughts and

emotions expressed by participants' think-aloud data to investigate the interaction condition between participants and GitHub Copilot. The second approach is based on the prompts written by participants. We designed a recovery process during which we apply minor changes to existing prompts and observe whether the refined prompts can generate the correct code that the original prompts failed to produce. We mainly focused on understanding how the scaffolding provided by the data scientists affected the code generated by GitHub Copilot. Additionally, we aimed to identify a potentially more efficient way for assisting data scientists in solving real-world problems.

4 RESULTS

In this section, we will discuss key observations recorded during the study. Out of the 15 tasks completed by the 5 participants, 6 tasks were successfully solved using the code generated by GitHub Copilot, while the remaining 9 tasks were considered failures due to either incorrect answers or non-workable code (detailed information in the appendix Table 2). By analyzing the prompts written by users and their think-aloud data, we were able to identify problems that hindered data scientists from effectively scaffolding prompts for GitHub Copilot to execute. The results demonstrate that not only factors from human and GitHub Copilot sides affect collaboration performance, but also the way in which human interacts with GitHub Copilot plays an important role in constructing efficient prompts.

4.1 Prompts Construction

The core step in collaborating with GitHub Copilot is constructing appropriate prompts that are understood by the AI model in order to generate functionally correct code. However, this is by no means an easy task. In our study, 5 out of 9 tasks failed due to improperly constructed prompts that were misinterpreted by GitHub Copilot. For each of these failed prompts, we reproduced the scenario and attempted to recover the prompts to generate workable code. Recovery details are shown in Table 1. By comparing the original prompts with the recovered prompts, several useful strategies can be inferred to assist data scientists in writing better prompts when collaborating with GitHub Copilot.

4.1.1 Choose Terminology Wisely. GitHub Copilot is essentially based on a Natural Language Processing model, which means that the choice of words and terminology in the user input can greatly influence the responses of the model. In our study, P4 encountered a filtering issue when performing data visualization in T3. His prompts could only generate code that messed up all column values in the same figure, as shown in Figure 1. In fact, all the columns were equivalent in that scenario, which indicates just selecting any of them would lead to the desired output.

We speculate that the word "rows" in the original prompts misled GitHub Copilot to assume that one row should contain multiple columns, which encouraged it to preserve the column information. Our recovered prompts replaced "rows" with "appearance", which is relatively more natural for humans to understand. Additionally, we noticed that GitHub Copilot demonstrates a certain level of robustness in translating natural language into domain-specific functions. For example, we changed "group by" to words "for each"

ID	Original Prompts	Recovered Prompts	Comments
P3T3	# calculate the sum of global sales of each game and whether the game appears more than once or not and then calculate the average of global sales of games appearing more than once and games appearing only once.	Not Recoverable	Prompts cannot be recovered unless using multiple steps with domain-specific words.
P4T1	# What is the result of dividing number of users who have a consecutive missing timestamp longer than 28 days by the total number of users?	Not Recoverable	Prompts cannot be recovered unless using multiple steps with domain-specific words.
P4T2	# Group by usg_apt and count the number of rows. Plot the number of rows in a bar chart.	# For each usg_apt, count the number of appearances. Plot the appearance in a bar chart.	Change Terminologies. Make use of robustness.
P5T2	# In df2, calculate number of rows of each usg_apt whose Year is from 2019 to 2022 # Then, for each year, find the top 10 usg_apt with the highest number of rows.	# In df2, calculate number of rows of each usg_apt whose Year is from 2019 to 2022. No need for type conversion. # Then, find the top 10 usg_apt with the highest number of rows for each year.	Change word order to avoid being left out by GitHub Copilot. Specify variable type.
P5T3	# In df3, for each Name, if it has multiple rows, find the sum of all Global Sales, and store them in new dataframe multi.	# In df3, for each Name, if it has multiple rows, find the sum of all Global Sales for each Name, and store them in new dataframe multi.	Change word order is not enough. Repeat the keywords to avoid being left out by GitHub Copilot.

Table 1: Failed Prompts and Corresponding Recovery

in T3, and it still resulted in performing a `groupby()` manipulation. A similar interpretation also occurred with “top 10”, which was translated to `nlargest()` automatically. Making use of this robustness could help programmers save time and effort. It can also assist programmers with little domain knowledge to perform relatively complex tasks.

4.1.2 Arrange the Word Order and Repeat Important Information.

Apart from the words themselves, the way in which words are arranged in sentences can also influence the meaning of the user input. The relative position of information within a sentence can emphasize different parts. Occasionally, GitHub Copilot may overlook important information in prompts and thus forget to perform certain manipulations. For example, P5 encountered one such problem where GitHub Copilot did not realize there was a `groupby()` manipulation, despite the presence of a “for each” indicator. As mentioned in the section above, GitHub Copilot has the robustness to translate “for each” to `groupby()`. However, it failed in this case. The original prompts placed the “for each year” information between the prefix “In df2” and the main body of the sentence. In contrast, our recovered prompts moved “for each year” to the end of the sentence as a suffix. We believe that during the preprocessing phase when sentences are broken down into pieces for sentiment

inference [13], GitHub Copilot might split the sentence in a way that the part of the prefix was discarded. A similar problem occurs again in P5T3. This time, we had to repeat the “for each” entry at both the prefix and the suffix position in order to notify GitHub Copilot about this “group by” step. To conclude, GitHub Copilot may overlook information due to improperly arranged word order in prompts. Moving ignored information or repeating it multiple times might help GitHub Copilot realize the necessary steps.

4.1.3 Think in a Programming Way.

To ensure effective communication with GitHub Copilot, users need to create prompts that are not only accurate and organized, but also easily comprehensible for GitHub Copilot. This includes using domain-specific terminologies, arranging words in a self-contained order, and constructing prompts in a more mathematical and quantitative way. For example, when doing T1, P5 wrote a prompt similar to the original prompt of P4 shown in Table 1. However, GitHub Copilot stopped generating code for this prompt. After we gave a hint that “length of missing timestamp” can also be described as “difference of two consecutive timestamps”, P5 immediately solved the task and said:

I feel like I’ve lost my programming ability. I didn’t describe the problem in a programming way. (P5)

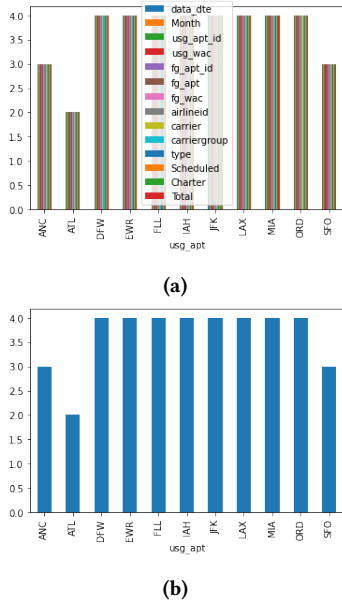


Figure 1: Original prompts messed up the visualization. We are only interested in the height of each bar (b). But the original prompts collapsed all useless column information into thin bars (a), severely distracting the visualization.

Since “calculate the difference” is actually a mathematical problem, “difference of two timestamps” is a more programming way to describe the tasks, while “missing timestamp” is a more natural language way. The programming way to construct a prompt will be easier for GitHub Copilot to understand.

4.2 Interaction between Human and GitHub Copilot

Another factor that leads to task failure is insufficient interaction between human and GitHub Copilot. Similar to collaboration among data scientists, working with GitHub Copilot requires recurrent iteration and refinement. A lack of interaction may lead to misunderstanding or credulity. In our study, 3 out of 9 failed tasks are caused by blindly trusting code generated by GitHub Copilot that appeared to be correct. Additionally, we also observed instances of under-trusting emotions during our study process.

4.2.1 Over-trust the GitHub Copilot. Compared with general-purpose programming, one significant difficulty for data science programming is determining the correctness of the output. Due to the complexity of the tasks involving large data sets and multiple steps of calculation and manipulation, it can be difficult to detect errors in the code. In the first task of our study, 3 out of 5 participants failed due to ignoring small bugs hidden in the generated code. For example, in this task, the data set should have first undergone a `groupby()` manipulation, followed by the calculation of differences between each “timestamp” value. However, although logically correct in a natural language sense, our participants’ prompts generated code with these two manipulations flipped, resulting in

treating inter-user differences as differences within each user group. Furthermore, since this error was implicit inside the data set, our participants did not suspect the correctness of the generated code and simply accepted it, yielding an incorrect final answer.

4.2.2 Under-trust the GitHub Copilot. There were also instances where participants struggled to write prompts but received no generated code from GitHub Copilot. This led to disappointment and frustration, causing participants to become overwhelmed by the negative attitudes and choose to give up on the task. During our study, P4 tried multiple prompts with various terminologies and word sequences for T1 but received no relevant response. He complained:

I am glad that I haven’t used GitHub Copilot yet. And I think I will not use it in the future. Because I find that describing a problem to GitHub Copilot is more difficult than coding up the problem by myself. Then why should I bother myself struggling with trying different prompts all the time? (P4)

He began to question the ability of GitHub Copilot to understand human language and even tried to write prompts like “What is 1+1?” to express his dissatisfaction. This experience can lead to a loss of confidence in the tool.

4.3 GitHub Copilot Flaw

During our study, we also discovered a limitation in GitHub Copilot’s functionality. It was found to ignore certain information in users’ prompts. For example, in T2, both P2 and P3 included “from 2019 to 2022” in their prompts. However, the code suggestions generated by GitHub Copilot ignored the specific time duration and applied the task to all years. Additionally, P3 explicitly requested GitHub Copilot to calculate the max from a list of values, but it did not include the `max()` function. Finally, P3 expressed frustration with this limitation:

It seems that GitHub Copilot can’t read my second sentence. (P3)

While some pieces of ignored information might be fixed by simply repeating them, others are difficult to fix and may lead to significant errors in the program. When P4 was working on T3, the prompt given is “Keep Global Sales, drop other columns”. However, the code suggested by GitHub Copilot dropped the “Global Sales” column instead. This might be due to GitHub Copilot ignoring words like “keep” and “other”, resulting in the code performing the opposite action.

5 DISCUSSION

5.1 Limitation

There are some limitations in this study. Firstly, the sample size is small. Only 5 people are willing to participate in the study. The small sample size may make our result inaccurate. Some findings may happen by accident, or there are some important things we ignored. Also, GitHub Copilot itself is not “stable”, which means it will give different code suggestions for the same prompt we give. When the prompt is given for the first time, GitHub Copilot generates code that doesn’t work. However, if we use another computer to write the prompt, or just delete it and write it again, GitHub Copilot may

give us a feasible code. This will also introduce some errors in our results and findings. Besides, we only point out some problems when users interact with GitHub Copilot, but we do not give some feasible solutions.

5.2 Future Work

Based on the limitations, future work should propose some feasible solutions on how to solve those problems we have raised. We may complete some extensions for GitHub Copilot to let it give users more information about how it generates the code or some feedback. For example, GitHub Copilot should highlight the keywords it used to give the code suggestion. In this way, users can know what happens inside GitHub Copilot so that they will not either over-trust it or under-trust it. Also, users can be more clear about which part of the information GitHub Copilot ignored.

6 CONCLUSION

This study reveals the current collaboration patterns between GitHub Copilot and data scientists, together with some problems that prevent more efficient communication and interaction for them. Scaffolding prompts for AI-based code generation tools is still one of the promising transformation directions for future data science. Conclusions and suggestions put forward by this work could provide some potentially constructive points for both data scientists and tool developers, leading toward better-developed data science community.

ACKNOWLEDGMENTS

Many thanks to April Wang for her devoted suggestions and guidance for this work.

REFERENCES

- [1] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 57 Morehouse Ln, Red Hook, NY 12571, 1877–1901. <https://proceedings.neurips.cc/paper/2020/file/1457c0d6bfc4967418bfb8ac142f64a-Paper.pdf>
- [2] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgren Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. <https://doi.org/10.48550/ARXIV.2107.03374>
- [3] Arghavan Moradi Dakhel, Vahid Majdinasab, Amin Nikanjam, Foutse Khomh, Michel C. Desmarais, Zhen Ming, and Jiang. 2022. GitHub Copilot AI pair programmer: Asset or Liability? <https://doi.org/10.48550/ARXIV.2206.15331>
- [4] GitHub. 2021. GitHub copilot - your AI pair programmer. <https://github.com/features/copilot>
- [5] SAS Institute Inc. 2014. What Makes a Great Data Scientist? https://www.sas.com/content/dam/SAS/en_gb/image/other1/events/WMagDS/DataScientist-survey-report-web%20FINAL.pdf
- [6] Nhan Nguyen and Sarah Nadi. 2022. An Empirical Evaluation of GitHub Copilot's Code Suggestions. In *Proceedings of the 19th International Conference on Mining Software Repositories (Pittsburgh, Pennsylvania) (MSR '22)*. Association for Computing Machinery, New York, NY, USA, 1–5. <https://doi.org/10.1145/3524842.3528470>
- [7] Foster Provost and Tom Fawcett. 2013. Data science and its relationship to big data and data-driven decision making. *Big data* 1, 1 (2013), 51–59.
- [8] Matt Przybyla. 2021. Data scientist vs computer scientist. here's the difference. <https://towardsdatascience.com/data-science-vs-computer-science-heres-the-difference-4b560de655f7>
- [9] Vineet Raina and Srinath Krishnamurthy. 2022. *Natural Language Processing*. Apress, Berkeley, CA, 63–73. https://doi.org/10.1007/978-1-4842-7419-4_6
- [10] Reid Robinson. 2023. How to write an effective GPT-3 prompt. <https://zapier.com/blog/gpt-3-prompt/?continueFlag=47e2137b96921456ad162513e61fc8e0>
- [11] Steven S. Skiena. 2017. *The Data Science Design Manual* (1st ed.). Springer Publishing Company, Incorporated, USA.
- [12] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. 2022. Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models. In *Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems* (New Orleans, LA, USA) (CHI EA '22). Association for Computing Machinery, New York, NY, USA, Article 332, 7 pages. <https://doi.org/10.1145/3491101.3519665>
- [13] Jonathan J. Webster and Chunyu Kit. 1992. Tokenization as the Initial Phase in NLP. In *Proceedings of the 14th Conference on Computational Linguistics - Volume 4* (Nantes, France) (COLING '92). Association for Computational Linguistics, USA, 1106–1110. <https://doi.org/10.3115/992424.992434>
- [14] Hadley Wickham, Mara Averick, Jennifer Bryan, Winston Chang, Lucy D'Agostino McGowan, Romain François, Garrett Grolemund, Alex Hayes, Lionel Henry, Jim Hester, et al. 2019. Welcome to the Tidyverse. *Journal of open source software* 4, 43 (2019), 1686.
- [15] Damir Yalalov. 2022. 100 best CHATGPT prompts to unleash AI's potential. <https://mpost.io/100-best-chatgpt-prompts-to-unleash-ai-potential/>

A TASK RESULTS

ID	Scenario	Time Used	Result	Comments
P1T1	S3	14'30"	Failure	Incorrect answer due to bugs
P1T2	S2	>15'	Failure	Incorrect answer due to bugs
P1T3	S1	4'20"	Success	
P2T1	S1	>15'	Failure	Incorrect answer due to bugs
P2T2	S3	6'50"	Success	
P2T3	S2	3'30"	Success	
P3T1	S2	>15'	Failure	Incorrect answer due to bugs
P3T2	S3	7'04"	Success	
P3T3	S1	>15'	Failure	Non-workable code
P4T1	S1	>15'	Failure	Non-workable code
P4T2	S2	>15'	Failure	Non-workable code
P4T3	S3	14'50"	Success	
P5T1	S2	14'55"	Success	
P5T2	S1	>15'	Failure	Non-workable code
P5T3	S3	>15'	Failure	Incorrect answer due to bugs

Table 2: 6 tasks succeeded in solving the problem, while 9 tasks failed to generate functionally correct code